
Computer Architecture and Organization

Serial-to-Parallel Converter Design

GitHub Link: <https://github.com/ssarakaya/serial-to-parallel-converter>

Sara Kaya - 22040102017

Digital Design (25 points)	Verilog Implementation (25 points)	Verilog Testbench (25 points)	Academic Report and Presentation (25 points)	Total (100 points)

	Student Name and Surname	Student ID	Signature
1			
2			
3			
4			
5			
6			
7			
8			

1. Introduction

Digital systems can transfer data either in serial form or parallel form. In serial communication, data bits are sent one after another through a single line. In parallel communication, multiple bits are transferred at the same time using multiple lines. Sometimes it is necessary to convert serial data into parallel data so that other digital components can process the information more efficiently.

In this project, a **Serial-to-Parallel Converter** is designed and implemented using **Verilog Hardware Description Language (HDL)**. The system receives serial data as input and converts it into a 4-bit parallel output. The functionality of the design is verified by creating a **Verilog testbench**, and the simulation results are observed using **GTKWave**.

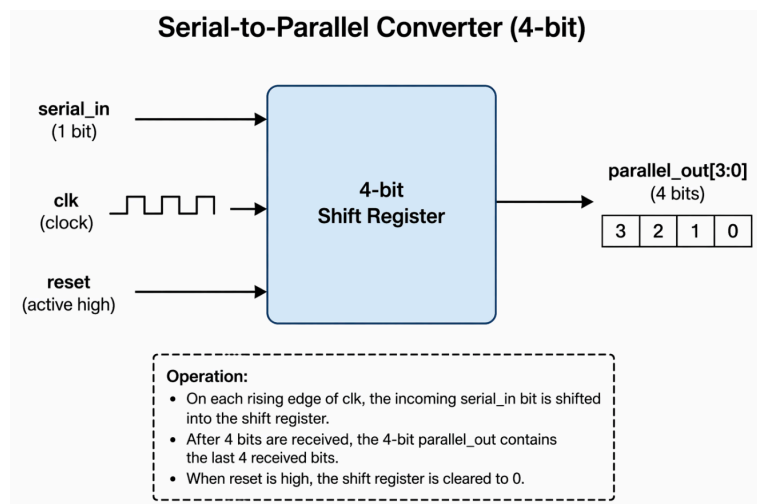
2. Digital Design

The main idea of the design is to use a **4-bit shift register**. A shift register stores data and shifts the stored bits at every clock cycle. In this design, the incoming serial bit is inserted into the shift register whenever a clock edge occurs.

The design uses the following signals:

- **clk** – clock signal that controls when data is shifted
- **reset** – clears the register and sets the output to zero
- **serial_in** – the serial data input
- **parallel_out[3:0]** – the 4-bit parallel output

When the reset signal is active, the shift register is cleared. After the reset is released, the incoming serial bits start entering the shift register. At every clock cycle, the previous bits move one position and the new bit is inserted into the register. After receiving four bits, the register contains the latest four bits and produces the parallel output.



3. Verilog Implementation

The hardware component was implemented using Verilog. The module receives the clock signal, reset signal, and serial input as inputs and produces a 4-bit parallel output.

The shift operation is implemented inside an always block that is triggered on the positive edge of the clock signal. When reset is active, the register is cleared. Otherwise, the register shifts and the new serial bit is added.

```

serial_to_parallel.v — Edited

module serial_to_parallel (
    input clk,
    input reset,
    input serial_in,
    output reg [3:0] parallel_out
);

always @(posedge clk or posedge reset)
begin
    if (reset)
        parallel_out <= 4'b0000;
    else
        parallel_out <= {parallel_out[2:0], serial_in};
end

endmodule

```

This implementation shifts the existing bits to the left and inserts the new serial input into the least significant position.

4. Verilog Testbench

To verify the correct operation of the module, a **testbench** was written in Verilog. The testbench generates clock signals and applies different serial input values to the module.

The clock signal toggles periodically, and the serial input is changed during simulation to test how the shift register behaves. The simulation allows us to observe how the serial bits are collected and converted into parallel output values.

```

serial_to_parallel_tb.v — Edited

`timescale 1ns/1ps

module serial_to_parallel_tb;

    reg clk;
    reg reset;
    reg serial_in;

    wire [3:0] parallel_out;

    serial_to_parallel uut (
        .clk(clk),
        .reset(reset),
        .serial_in(serial_in),
        .parallel_out(parallel_out)
    );

    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, serial_to_parallel_tb);

        clk = 0;
        reset = 1;
        serial_in = 0;

        #10 reset = 0;

        #10 serial_in = 1;
        #10 serial_in = 0;
        #10 serial_in = 1;
        #10 serial_in = 1;

        #40 $finish;
    end

    always #5 clk = ~clk;

endmodule

```

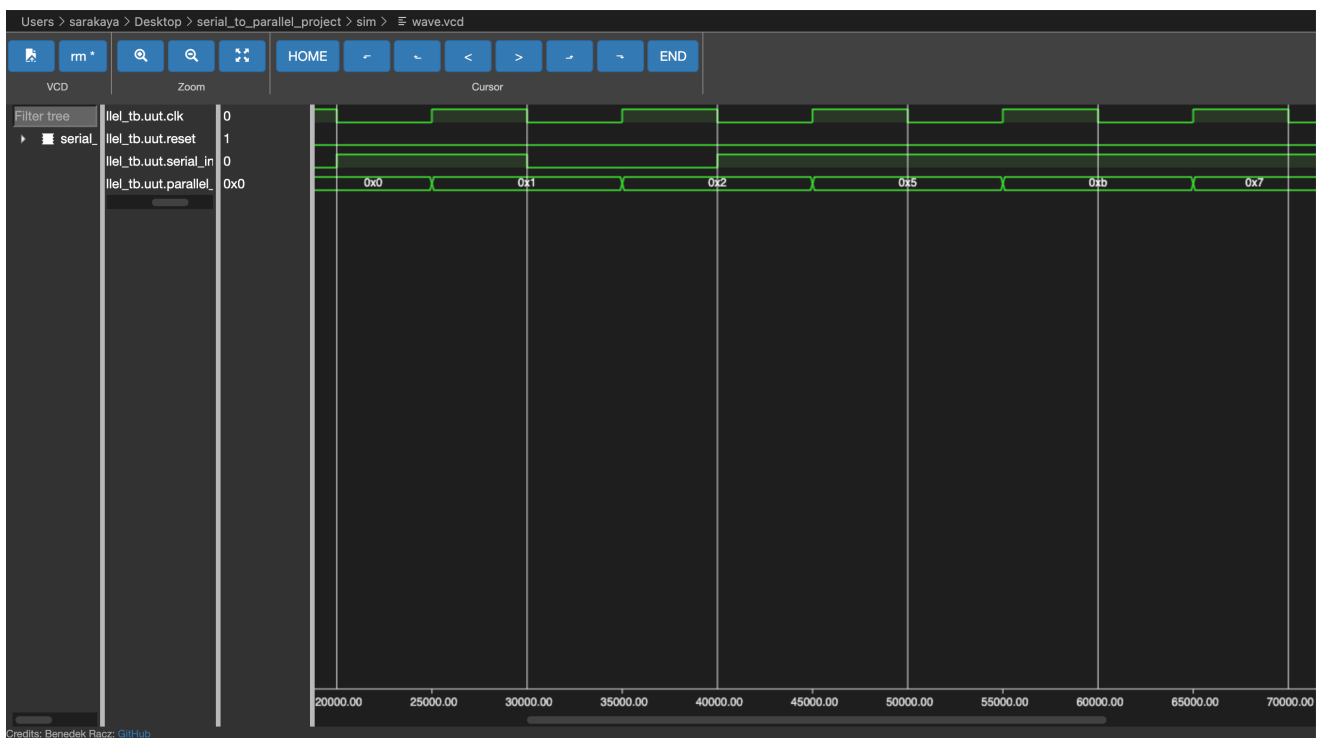
The testbench provides a sequence of serial bits and observes the resulting parallel output.

5. Simulation Results

The design was simulated and the waveform results were examined using **GTKWave**. The waveform shows the clock signal, reset signal, serial input, and the 4-bit parallel output.

At the beginning of the simulation, the reset signal clears the register and the output becomes zero. After the reset signal is disabled, serial bits start entering the shift register. At every clock cycle, the bits shift and eventually form the 4-bit parallel output.

The waveform confirms that the converter correctly collects serial bits and produces the expected parallel output values.



6. Conclusion

In this project, a Serial-to-Parallel Converter was successfully designed using Verilog HDL. The system uses a 4-bit shift register to collect incoming serial bits and convert them into parallel output data.

The functionality of the design was verified using a Verilog testbench and the simulation results observed in GTKWave confirmed that the design works correctly. This project demonstrates how hardware description languages can be used to design and test digital hardware components.